Homework 4.3

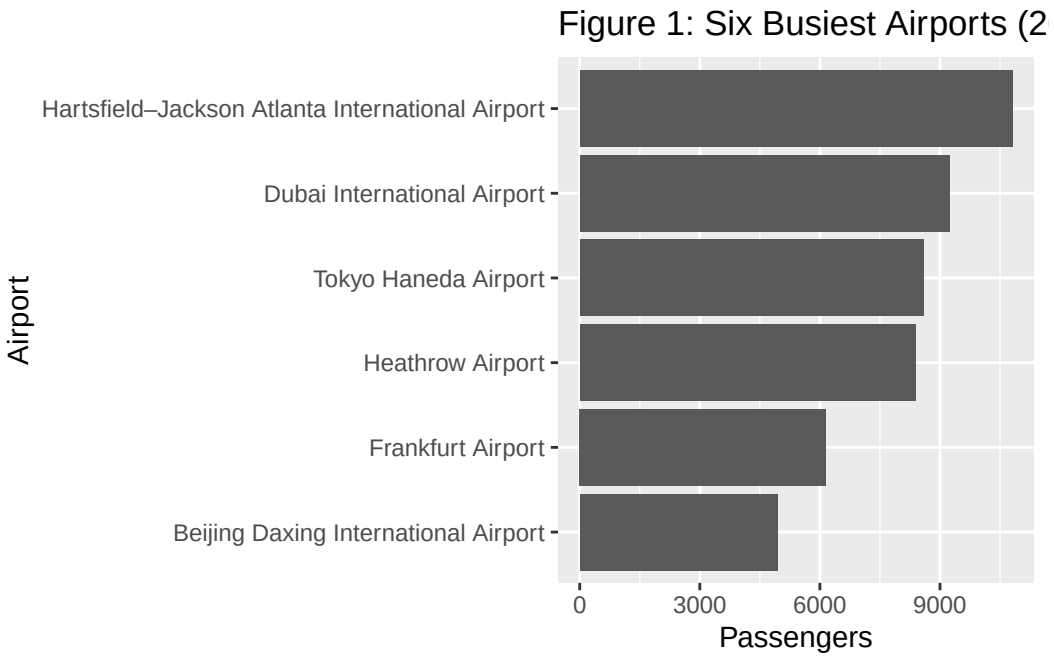
Timilehin Balogun

Busiest Airports Analysis

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

Table 1: Six Busiest Airports (2025)

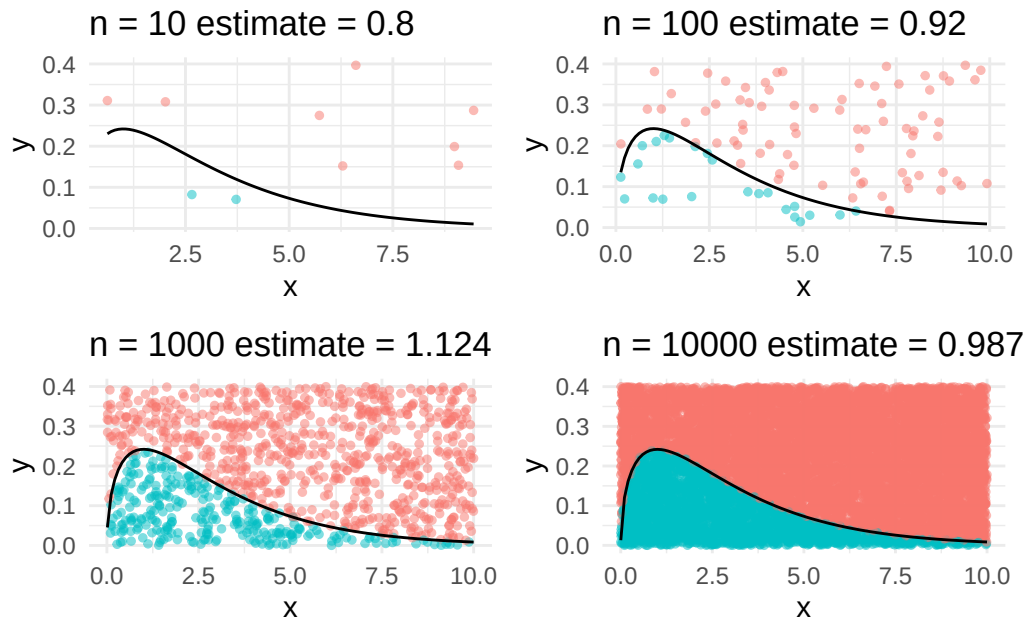
Airport	Passengers (Millions)
Hartsfield–Jackson Atlanta International Airport	10806.777
Dubai International Airport	9233.151
Tokyo Haneda Airport	8590.017
Heathrow Airport	8388.457
Frankfurt Airport	6156.496
Beijing Daxing International Airport	4944.103



This table and bar chart compare the traffic across 6 different "busy" airports. It shows that the airport that's the busiest is Beijing Daxing Int. Airport.

Monte Carlo Integration

Figures 2-5



Planning and Prompting Gen AI

Plan for Tidying and Visualization

Prompt: Using this plan , help me tidy the calcium dataset and create a plot in R using ggplot2.

The dataset is not tidy. The first four columns are the null group and the next four columns are the treatment group. Each column represents time points .

Follow the plan I paste to: 1. tidy the dataset 2. create a plot comparing calcium levels over time between groups

Plan That followed:

Goal:
The goal is to tidy the calcium dataset and create a plot comparing calcium levels over time between the null and treatment groups.

Needs:
- Load dataset
- Convert to tidy format
- Create group variable

- Create time variable
- Use ggplot

Steps:
1. Load dataset
2. Identify null vs treatment columns
3. Convert from wide to long format
4. Create group variable
5. Create time variable
6. Check tidy structure
7. Create plot comparing groups over time

Copilot Response:

Below is a compact R script that loads your calcium.csv, tidies it (wide → long), creates group and time variables, checks the tidy structure, and makes a clear ggplot comparing null vs treatment across times $0 - 3$. Copy–paste into an R session (or RStudio) and run.

Notes and minimal guidance File path: If calcium.csv is not in your working directory, give the full path to read.csv().

Missing values: The script uses values_drop_na = TRUE so rows with missing measurements are ignored in the long table and summaries.

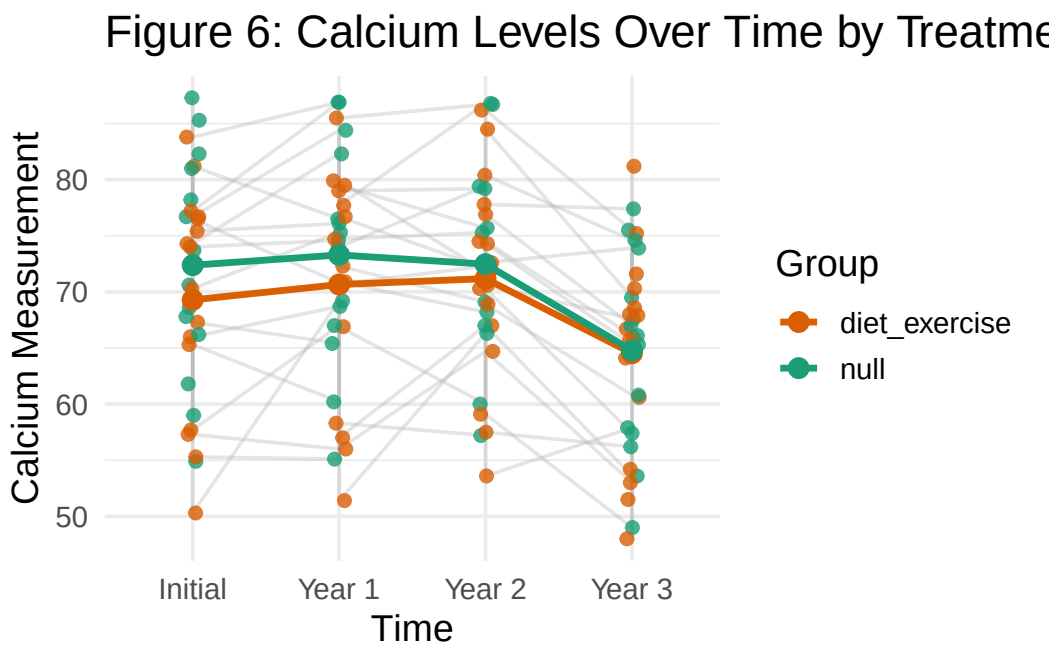
Column naming: The code assumes the first four columns are the null group and the next four are treatment (as your header repeats y0..y3 twice). If your actual grouping is different, swap the naming vector accordingly.

Plot tweaks: To show individual sample points instead of aggregated means, replace the plotting data with df_long and use geom_jitter() or geom_point(alpha=0.4).

If you want, I can (1) produce a version that plots all individual samples plus the group mean, or (2) save the plot to a PNG file — tell me which and I'll keep it minimal.

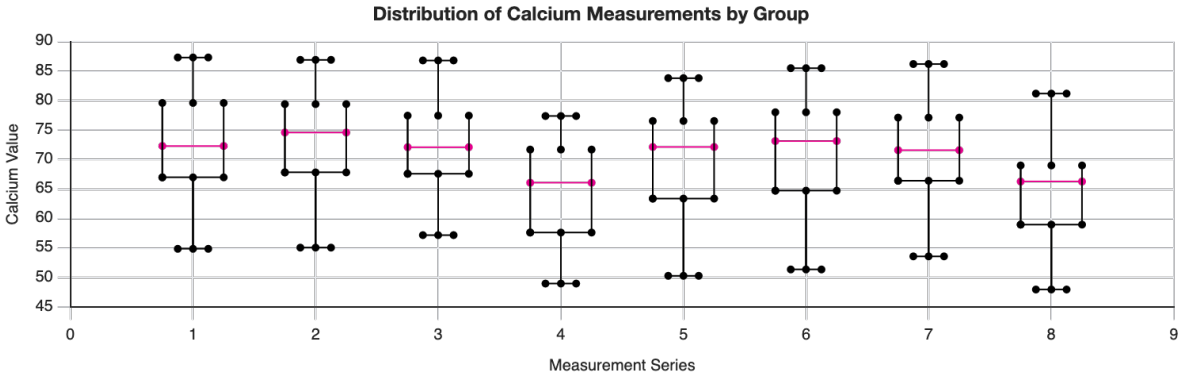
```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.1     v stringr   1.6.0
v lubridate 1.9.5     v tibble    3.3.1
v purrr     1.2.1     v tidyr     1.3.2
v readr     2.2.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter()         masks stats::filter()
x readr::guess_encoding() masks rvest::guess_encoding()
x dplyr::lag()            masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
New names:

[1] "First 12 rows of tidy dataframe:"
```

```
# A tibble: 12 x 4
      id group         time    calcium
   <int> <chr>         <ord>     <dbl>
 1     1 null          Initial    87.3
 2     1 null          Year 1     86.9
 3     1 null          Year 2     86.7
 4     1 null          Year 3     75.5
 5     1 diet_exercise Initial    83.8
 6     1 diet_exercise Year 1     85.5
 7     1 diet_exercise Year 2     86.2
 8     1 diet_exercise Year 3     81.2
 9     2 null          Initial    59
10     2 null          Year 1     60.2
11     2 null          Year 2     60
12     2 null          Year 3     53.6

[1] "Shape of tidy dataframe: 124 rows, 4 columns"

[1] "Summary statistics by group and time:"

# A tibble: 8 x 5
  group         time    mean_calcium sd_calcium     n
  <chr>         <ord>          <dbl>      <dbl> <int>
1 diet_exercise Initial         69.3       9.91    16
2 diet_exercise Year 1          70.7      10.0     16
3 diet_exercise Year 2          71.2       9.29    16
4 diet_exercise Year 3          64.5       9.02    16
5 null          Initial         72.4       9.60    15
6 null          Year 1          73.3       9.44    15
7 null          Year 2          72.5       8.48    15
8 null          Year 3          64.8       8.69    15

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```



Generic Response:

Figure 7



Visualization Created: Boxplot of Calcium Measurements The figure below is a box-and-whisker plot showing the distribution of calcium values for each measurement series (y0, y1, y2, y3, repeated across groups). Why a boxplot? A boxplot is well suited for this dataset because it: • Shows central tendency (median) • Displays spread and variability (quartiles and range) • Highlights outliers • Allows easy comparison across multiple groups What you can see • Differences in median calcium levels across measurement series • Variation within each series • Potential outliers or unusually low/high measurements The visualization

If you'd like, I can also: • Create line plots (e.g., trends across rows) • Aggregate repeated variables (y0–y3) and visualize means ± SD • Produce publication-ready figures (journal style, grayscale, higher resolution) • Export the visualization to PNG, PDF, or SVG Just tell me how you plan to use it (exploratory analysis, presentation, or publication).

Compared to mine, copilot did follow my plan and was able to generate the code, but the generation of the plot to visualize it was not as good as with the generic prompt. I feel the simplicity of the generic prompt allowed it to fully understand what it was that it needed to do, while mine made it complicate things and build confusing plots. All in all, this shows that if the plan given to an AI model is clear and simple, it'll likely do better than if it's long and convoluted.

Reflection

I have learned a lot in R this semester. I have been able to learn how to scrape data from websites and how to clean it and filter it. That was shown in the first example of work done here, concerning the airports. I have also learnt how to create data visualizations using ggplot and others like i did in the monte carlo work. Lastly, I have also learned, among many other things, how to use genAI to help with my code and tidyness.

Appendix:

Microsoft Copilot, Windows 11

Date: 4/15/2026

Version: Free

Prompt: Using this plan , help me tidy the calcium dataset and create a plot in R using ggplot2.

The dataset is not tidy. The first four columns are the null group and the next four columns are the treatment group. Each column represents time points .

Follow the plan I paste to: 1. tidy the dataset 2. create a plot comparing calcium levels over time between groups

Plan That followed:

Goal:
The goal is to tidy the calcium dataset and create a plot comparing calcium levels over time between the null and treatment groups.

Needs:
- Load dataset
- Convert to tidy format
- Create group variable
- Create time variable
- Use ggplot

Steps:
1. Load dataset
2. Identify null vs treatment columns
3. Convert from wide to long format
4. Create group variable
5. Create time variable
6. Check tidy structure
7. Create plot comparing groups over time

Copilot Response:

Below is a compact R script that loads your calcium.csv, tidies it (wide → long), creates group and time variables, checks the tidy structure, and makes a clear ggplot comparing null vs treatment across times $0 - 3$. Copy–paste into an R session (or RStudio) and run.

Notes and minimal guidance File path: If calcium.csv is not in your working directory, give the full path to read.csv().

Missing values: The script uses values_drop_na = TRUE so rows with missing measurements are ignored in the long table and summaries.

Column naming: The code assumes the first four columns are the null group and the next four are treatment (as your header repeats y0..y3 twice). If your actual grouping is different, swap the naming vector accordingly.

Plot tweaks: To show individual sample points instead of aggregated means, replace the plotting data with df_long and use geom_jitter() or geom_point(alpha=0.4).

If you want, I can (1) produce a version that plots all individual samples plus the group mean, or (2) save the plot to a PNG file — tell me which and I'll keep it minimal.

Code Used:

BUSIEST AIRPORTS ANALYSIS

library(rvest) library(dplyr) library(ggplot2)

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic" page <- read_html(url) tables <- html_table(page, fill = TRUE)

airports_raw <- tables[[2]]

airports_raw$Totalpassengers < -as.numeric(gsub("[^0-9]", "", airports_raw$Totalpassengers)) / 1e6

airports_selected <- airports_raw %>% filter(Airport %in% c(“Hartsfield–Jackson Atlanta International Airport”, “Frankfurt Airport”, “Beijing Daxing International Airport”, “Dubai International Airport”, “Tokyo Haneda Airport”, “Heathrow Airport”))

knitr::kable(airports_selected[, c(“Airport”, “Totalpassengers”)], col.names = c(“Airport”, “Passengers (Millions)”), caption = “Six Busiest Airports (2025)”)

ggplot(airports_selected, aes(x = reorder(Airport, Totalpassengers), y = Totalpassengers)) + geom_col() + coord_flip() + labs(title = “Figure 1: Six Busiest Airports (2025)”, x = “Airport”, y = “Passengers”)

MONTE CARLO INTEGRATION

library(ggplot2) library(patchwork)

set.seed(1)

f <- function(x){ dchisq(x, df = 3) }

mc_plot <- function(n){

x <- runif(n, 0, 10) y <- runif(n, 0, 0.4)

df <- data.frame(x, y)

df$below < -df$y < f(df$x)

est <- mean(df$below) * (10 * 0.4)

ggplot(df, aes(x, y)) + geom_point(aes(color = below), size = 1, alpha = 0.5) + stat_function(fun = f) + labs(title = paste(“n =”, n, “estimate =”, round(est, 3)), x = “x”, y = “y”) + theme_minimal() + theme(legend.position = “none”) }

p1 <- mc_plot(10) p2 <- mc_plot(100) p3 <- mc_plot(1000) p4 <- mc_plot(10000)

(p1 + p2) / (p3 + p4)

CALCIUM TIDYING + COPILOT RESPONSE

library(tidyverse)

df_raw <- read_csv("calcium.csv", col_names = TRUE, show_col_types = FALSE)

colnames(df_raw) <- c("null_y0", "null_y1", "null_y2", "null_y3", "treat_y0", "treat_y1", "treat_y2", "treat_y3")

df_raw <- df_raw %>% mutate(id = row_number())

tidy_df <- df_raw %>% pivot_longer(cols = starts_with(c("null_", "treat_")), names_to = "group_time", values_to = "calcium") %>% filter(!is.na(calcium)) %>% separate(group_time, into = c("group", "time_code"), sep = "_") %>% mutate(group = if_else(group =="null", "null", "diet_exercise"), time = case_when(time_code == "y0" ~ "Initial", time_code == "y1" ~ "Year 1", time_code == "y2" ~ "Year 2", time_code == "y3" ~ "Year 3", TRUE ~ time_code), time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3"), ordered = TRUE)) %>% select(id, group, time, calcium)

summary_stats <- tidy_df %>% group_by(group, time) %>% summarise(mean_calcium = mean(calcium), sd_calcium = sd(calcium), n = n(), .groups = "drop")

library(ggplot2)

p <- ggplot(tidy_df, aes(x = time, y = calcium, color = group)) + geom_line(aes(group = id), color = "gray70", alpha = 0.35) + geom_point(position = position_jitter(width = 0.05, height = 0), size = 1.8, alpha = 0.8) + stat_summary(aes(group = group, color = group), fun = mean, geom = "line", size = 1.2) + stat_summary(aes(group = group, color = group), fun = mean, geom = "point", size = 3) + scale_color_manual(values = c("null" = "#1b9e77", "diet_exercise" = "#d95f02")) + labs(title = "Figure 6: Calcium Levels Over Time by Treatment Group", x = "Time", y = "Calcium Measurement", color = "Group") + theme_minimal(base_size = 14) + theme(legend.position = "right")

ggsave("calcium_trends.png", plot = p, width = 9, height = 6, dpi = 300)

print(p)